

L'extension *intexgral**

Valentin Dao[†]

Publiée le 26-07-2025

Résumé

Tandis que la composition d'intégrales est très fréquente en $\text{\LaTeX}$, cette dernière s'avère souvent peu pratique. Lorsque l'expression se complexifie, il devient alors difficile d'en modifier les différents éléments dans un code source peu lisible. Pour répondre à ce problème, le package *intexgral* met à disposition une macro centrale dont le seul argument est l'intégrande. Tout le reste (les symboles, les bornes, les variables d'intégration...) pourra aisément être modifié grâce à une interface $\langle \text{clé}=\text{valeur} \rangle$. Contrairement à la méthode classique, où l'utilisateur choisit l'ordre des bornes avec les caractères actifs $_$ et $^$, le package a dû fixer une convention. Ainsi, pour les deux clés gérant les bornes qui seront présentées, l'entrée supposée sera $_{\langle \text{borne inférieure} \rangle}^{\langle \text{borne supérieure} \rangle}$. Ce package étant écrit en *expl3*, il dépend donc des packages $\text{\LaTeX3}$ *l3kernel* et *l3package*. De plus, *intexgral* utilise le package *derivative*¹ afin de faciliter la manipulation de différentielles, ainsi que le package *pkginfograb*².

*Ce fichier décrit la version v3.0.0, mise à jour le 24-12-2025

[†]E-mail : vdao.texdev@gmail.com

1. Disponible sur CTAN sous : <https://ctan.org/pkg/derivative>

2. Disponible sur CTAN sous : <https://ctan.org/pkg/pkginfograb>

Table des matières

Résumé	1
1 Options de package	4
2 Présentation de la macro principale	4
3 Liste des clés	5
3.1 Bornes d'intégration	5
3.1.1 Définir les bornes	5
3.2 Modes d'affichage	6
3.3 Symbole (et encore des bornes)	7
3.3.1 Sélectionner le symbole	7
3.3.2 Générer beaucoup d'intégrales	7
3.3.3 Définir des bornes ponctuellement	8
3.3.4 Motifs récurrents de bornes	8
3.4 Différentielles	9
3.4.1 Spécifier les variables d'intégration	9
3.4.2 Inclure le jacobien	9
3.4.3 Modifier le symbole des différentielles	10
3.4.4 Réaliser une intégrale curviligne	10
3.4.5 Opter pour la variante étoilée	10
3.4.6 Désigner des options	10
4 Macros annexes	11
4.1 Emplacements personnalisés des différentielles	11
4.2 Retour sur la clé <i>limits</i>	12
4.3 Retour sur la clé <i>variables</i>	13
4.4 Retour sur la clé <i>symbol</i>	14
5 Syntaxe <i>spéciale</i>	15
5.1 Présentation de la syntaxe	15
5.2 Fonctionnement de la syntaxe	16
6 Paramètres annexes	16
Historique des modifications	18
Index	20
Implémentation	26

Licence

© 2025 Valentin Dao, publié sous la $\text{\LaTeX}$ Project Public License (LPPL) 1.3c

Polices

Chronicle Text G3 Roman

© 2002, 2007 Hoefler & Frere-Jones.

Whitney-Medium

© 1996, 2009 Hoefler & Frere-Jones.

Monospace Argon Medium

© 2023, GitHub <https://github.com/githubnext/monospace>

Dépôt

 [Voir dépôt GitHub](https://github.com).

Remerciements

Un grand merci à Plante et Slurpy de m'avoir accompagné dans cette aventure que fut l'apprentissage de $\text{\TeX}$, vos conseils ont été précieux. Je tiens également à remercier Anthony pour avoir toujours gentiment révisé les nouvelles versions et m'avoir donné des idées pour celles à venir.

1 Options de package

`\intexgralsetup` `\intexgralsetup` {<options de package>}

nouveau: v2.0.0

Ces options peuvent être déclarées de façon classique avec `\usepackage` ou bien avec la macro ici présente dans le préambule :

`invert-limits` `invert-limits=<true|false>`

Cette clé permet d'inverser la convention d'ordre des limites. Le document ne peut suivre qu'une seule convention.

`invert-diff` `invert-diff=<true|false>`

mise à jour: v3.0.0

Il est courant de voir les variables d'intégration placées avant l'intégrande dans les articles de physique. Cette clé intervertit donc leurs positionnements.

`limits-mode` `limits-mode=<limits|nolimits>`

nouveau: v3.0.0

Applique `\limits` ou `\nolimits` à toutes les intégrales.

`italic` `italic=<true|false>`

`upright` `upright=<true|false>`

Ce sont les deux clés du package *derivative*.

2 Présentation de la macro principale

`\integral` `\integral` [<list de clés>] {<intégrande>}

mise à jour: v3.0.0

Cette macro est celle qui permet de composer les intégrales. Elle doit naturellement être utilisée en mode mathématique uniquement. En voici un premier exemple sous sa forme la plus simple :

```
\begin{equation}
  \integral{x}
\end{equation}
```

$$\int x \, dx \tag{1}$$

Puisque cette macro est axée autour de l'utilisation de clés, la première partie de cette documentation les présentera en les regroupant par domaines. La seconde partie quant à elle introduira les quelques macros annexes qui viennent compléter l'usage de certaines clés. Le reste de ce document exposera les autres fonctionnalités du package.

3 Liste des clés

3.1 Bornes d'intégration

3.1.1 Définir les bornes

`limits` `limits= {⟨liste-mixte⟩}, ⟨mot-clé⟩`
`limits*` `limits*= {⟨liste-mixte⟩}, ⟨mot-clé⟩`

Cette clé détermine les bornes d'intégration à utiliser en suivant la convention édictée dans le résumé. Sous sa forme la plus simple, la valeur de la clé prend comme argument une liste d'éléments séparés par une virgule (*comma-separated values* ou `⟨csv-list⟩`¹).

```
\begin{equation}
  \integral[limits={1, 10}]{f(x)}
\end{equation}
```

$$\int_1^{10} f(x) \, dx \quad (2)$$

L'avantage notable de la clé `limits` est qu'on peut lui spécifier les bornes d'intégration de plusieurs intégrales, et le package compose automatiquement les symboles correspondants. Pour séparer les paires de bornes, il faut utiliser le point-virgule (*semicolon-separated values* ou `⟨ssv-list⟩`¹).

```
\begin{equation}
  \integral[limits={1, 2; 3, 4}, variables={x, y}]{f(x, y)}
\end{equation}
```

$$\int_1^2 \int_3^4 f(x, y) \, dx \, dy \quad (3)$$

Il est également possible de renseigner des bornes prédéfinies à l'aide de mots-clés (voir section 4.2). Enfin, la variante étoilée permet d'exprimer les bornes de l'intégrale sous forme d'intervalle. Le cas échéant, le sens des crochets s'adapte automatiquement à la présence de $\pm\infty$

```
\begin{equation}
  \integral[limits*={1, 10}]{f(x)}
\end{equation}
```

$$\int_{[1,10]} f(x) \, dx \quad (4)$$

1. On entend donc par `⟨liste-mixte⟩` le fait que `limits` puisse accepter un ensemble de `⟨csv-list⟩` dans une `⟨ssv-list⟩` plus globale.

3.2 Modes d'affichage

mode `mode=\langle\text{default}|\text{nested}|\text{product}\rangle`

nouveau: v3.0.0 *Uniquement* lorsque la clé `limits` est employée, il est possible d'alterner entre trois styles d'affichage distincts. Chacun d'entre eux est entièrement compatible avec l'option `invert-diff`.

default Compose l'ensemble des symboles, puis l'intégrande, puis les variables d'intégration. La macro opérant par défaut dans ce mode, il n'est donc pas nécessaire d'utiliser la clé avec cette valeur.

nested Compose une intégrale *imbriquée* où symbole et intégrande alternent avant que tous les variables d'intégrations soient écrits.

product Compose un produit d'intégrales où symbole, intégrande et variables d'intégration alternent.

Bien sûr, rien ne vous empêche pour le mode produit d'utiliser plusieurs fois la macro `\integral` d'affilée pour produire le même résultat que dans l'exemple ci-dessous. Pour celles et ceux qui préféreraient cependant obtenir le résultat avec une seule macro, cette méthode est permise.

important: Pour les deux derniers modes, l'intégrande sera, d'une certaine manière, *découpée*. Afin de correctement effectuer cette action, il faut avoir recours à la même méthode qu'avec `limits`, c'est-à-dire en employant le point-virgule.

MODE **default**

```
\begin{equation}
  \integral[limits={1, 2; 3, 4; 5, 6}, variables={x, y, z}, mode=default]{xyz}
\end{equation}
```

$$\int_1^2 \int_3^4 \int_5^6 xyz \, dx \, dy \, dz \quad (5)$$

MODE **nested**

```
\begin{equation}
  \integral[limits={1, 2; 3, 4; 5, 6}, variables={z, y, x}, mode=nested]{x;y;z}
\end{equation}
```

$$\int_1^2 x \int_3^4 y \int_5^6 z \, dz \, dy \, dx \quad (6)$$

MODE **product**

```
\begin{equation}
  \integral[limits={1, 2; 3, 4; 5, 6}, variables={x, y, z}, mode=product]{x;y;z}
\end{equation}
```

$$\int_1^2 x \, dx \int_3^4 y \, dy \int_5^6 z \, dz \quad (7)$$

Pour bien fonctionner, il est évident que les clés `limits` et `variables` (voir 3.4.1), ainsi que l'intégrande, doivent avoir le même nombre d'éléments. Si ce n'est pas le cas, la compilation n'échouera pas, mais l'expression de l'intégrale risque de ne plus avoir aucun sens. De nombreux messages d'avertissement seront là pour vous en informer.

3.3 Symbole (et encore des bornes)

Les clés `limits(*)` se révèlent très utiles lorsque l'on souhaite composer une intégrale définie. Néanmoins, cela couvre seulement les expressions finales où les bornes de chaque variable ont été spécifiées. Pour des cas plus généraux — les intégrales indéfinies donc — elles sont relativement malcommodes. Pour composer une intégrale double sur une surface S , on devrait écrire `limits={, ; S, }`. Il va sans dire que c'est assez mal adapté pour l'utilisateur et peu optimal pour le package. Par ailleurs, le glyphe ne serait pas correct. L'ensemble des clés à suivre propose donc une façon de facilement modifier le symbole ainsi que les bornes.

3.3.1 Sélectionner le symbole

`symbol`

`symbol`=⟨séquence de contrôle⟩

mise à jour: v3.0.0

Cette clé accepte une macro désignant un symbole d'intégration. Tout symbole préalablement défini par une séquence de contrôle est recevable. Si vous tentez d'en utiliser une qui n'est pas définie, elle sera substituée à `\iint` et un message d'avertissement sera émis.

remarque : À part la vérification de l'existence de la macro, aucun contrôle particulier n'est effectué et la valeur de la clé est utilisée telle quelle pour composer le symbole. Ceci implique notamment deux choses. Tout d'abord, le symbole va dépendre de la façon dont il est défini. Par exemple, `amsmath` et `unicode-math` ne définissent pas `\iint` de la même manière. Le résultat sera donc différent. De plus, si vous renseignez comme argument de la clé une macro *définie* mais ne correspondant pas à un symbole intégral, le package n'en tiendra pas compte et utilisera quand même le symbole donné. Outre l'incohérence mathématique que cela peut représenter, il est susceptible que cela mène, sous certaines circonstances, à des erreurs de bas niveau.

```
\begin{equation}
  \integral[symbol=\iint, llimit=S, variables={x, y}]{f(x, y)}
\end{equation}
```

$$\iint_S f(x, y) \, dx \, dy \quad (8)$$

3.3.2 Générer beaucoup d'intégrales

`nint` `nint`=⟨entier⟩

Cette clé accepte un entier n qui permet de composer n intégrales. Il est conseillé d'avoir recours à cette clé seulement si le nombre de symboles dépasse 4 afin de privilégier les glyphes définis par la police mathématique utilisée.

```
\begin{equation}
\int\limits[nint=5, llimit=\Omega, variables={x_1, x_2, x_3, x_4, x_5}]{f(x_1, x_2,
x_3, x_4, x_5)}
\end{equation}
```

$$\iiint\limits_{\Omega} f(x_1, x_2, x_3, x_4, x_5) dx_1 dx_2 dx_3 dx_4 dx_5 \quad (9)$$

3.3.3 Définir des bornes ponctuellement

```
llimit llimit=<borne inférieure>
ulimit ulimit=<borne supérieure>
```

mise à jour: v3.0.0 Ces deux clés permettent de spécifier les bornes inférieures et supérieures respectivement. Elles ne sont adaptées que si un seul symbole est affiché. Si vous avez besoin de spécifier les deux bornes, la clé `limits` devra être privilégiée.

```
\begin{equation}
\int\limits[llimit={x^2 + y^2 \leq 1}, variables={x, y}]{f(x, y)}
\end{equation}
```

$$\int_{x^2+y^2 \leq 1} f(x, y) dx dy \quad (10)$$

3.3.4 Motifs récurrents de bornes

Les bornes suivent parfois des schémas courants qui peuvent se généraliser avec des clés, évitant ainsi de surcharger l'argument de `llimit`.

```
domain domain= {<*-liste>}
domain* domain*= {<*-liste>}
```

mise à jour: v3.0.0 Ces clés acceptent une liste dont le délimiteur est un astérisque. Ensuite, chaque élément de la liste est analysé de la façon suivante :

- Le premier token de l'item se voit passer `\mathbb` (précédé d'`\uppercase` au cas où la touche SHIFT serait trop loin pour vos doigts).
- Les tokens restants — qui peuvent être vides — sont placés comme exposant (ou en indice pour la variante étoilée de la clé).

```
\begin{equation}
\int\limits[symbol=\iint, domain={\mathbb R*\mathbb R}, variables={x, y}]{xy}
\end{equation}
```

$$\iint_{\mathbb R \times \mathbb R} xy dx dy \quad (11)$$

```
boundary boundary= {<borne inférieure>}
```

Cette clé place simplement le symbole ∂ avant la borne inférieure.


```
\begin{equation}
  \integral[symbol=\oint, boundary=S, diff-vec]{G(\vec r)}
\end{equation}
```

$$\oint_{\partial S} G(\vec{r}) \cdot d\vec{r} \quad (12)$$

3.4 Différentielles

3.4.1 Spécifier les variables d'intégration

variables `variables={⟨liste-csv⟩, ⟨mot-clé⟩, none`

mise à jour: v3.0.0

Cette clé permet de définir les variables d'une intégrale sous forme d'une `⟨csv-list⟩`. La clé peut, tout comme `limits`, accepter un mot-clé comme argument. Ce comportement est aussi expliqué plus tard (voir section 4.3)

remarque : Si aucune variable d'intégration n'est renseignée, c'est-à-dire que `variables` n'est pas appelée, le package place automatiquement «*dx*» (ou «*d* $\vec{r}$ » si `diff-vec` est actif). De plus, si `none` est passé comme valeur, aucune variable d'intégration ne sera affichée.

```
\begin{equation}
  \integral[variables=t]{t^2}
\end{equation}
```

$$\int t^2 dt \quad (13)$$

3.4.2 Inclure le jacobien

jacobian `jacobian`

Cette clé active l'affichage du jacobien lorsque défini par `\NewVariableKeyword`.

```
\begin{equation}
  \integral[limits={0, R; 0, 2\pi; 0, \pi}, variables=spherical, mode=product,
  jacobian]{;}
\end{equation}
```

$$\int_0^R r^2 dr \int_0^{2\pi} \sin \theta d\theta \int_0^\pi d\phi \quad (14)$$

Comme évoqué plus tôt, `derivative` est utilisé par le package pour composer les différentielles. L'ensemble des clés à suivre, dont le nom portera le préfixe «*diff-*»¹, vous permettent ainsi d'appliquer les fonctionnalités de la macro `\odiff`.

1. Quoique la clé `diff-vec` ne soit pas liée à `derivative`.

3.4.3 Modifier le symbole des différentielles

diff-symb diff-symb= $\langle$ macro de différentielle $\rangle$

Cette clé permet de modifier le style des différentielles parmi ceux définis par `\NewDifferential`.

```
\NewDifferential{\feynmandiff}{\mathcal{D}}
\begin{equation}
\integral[variables={q(t)}, diff-symb=\feynmandiff]{e^{+\dfrac{\imath}{\hbar} S[q(t)]}}{
\hbar}
\end{equation}
```

$$\int e^{+\frac{iS[q(t)]}{\hbar}} \mathcal{D}q(t) \quad (15)$$

3.4.4 Réaliser une intégrale curviligne

diff-vec diff-vec

Cette clé applique un vecteur à chaque variable d'intégration en plus d'un point médian. Elle n'est compatible qu'avec le mode `default`.

```
\begin{equation}
\int[double=S, variables=S, diff-vec]{\vec F}
\end{equation}
```

$$\iint_S \vec{F} \cdot d\vec{S} \quad (16)$$

3.4.5 Opter pour la variante étoilée

diff-star diff-star

Activer cette clé revient à utiliser `\odif*` ou ses variantes.

```
\begin{equation}
\int[double, variables={x, y}, diff-star]{x^2 \exp(y)}
\end{equation}
```

$$\iint x^2 \exp(y) \, \mathrm{d}_x \mathrm{d}_y \quad (17)$$

3.4.6 Désigner des options

```
diff-options diff-options= {<liste clé-valeur>}
```

Cette clé accepte une liste de clés telle qu'elle aurait été écrite comme argument optionnel de `\odif` ou ses variantes. Pour illustrer, `diff-options={order={2, 3}, var=all}` sera interprété comme `\odif[order={2, 3}, var=all]{...}`.

```
\begin{equation}
\integral[diff-options={order=4}]{\bar{\psi}(x)(\imath\gamma^\mu\partial_\mu-m)\psi(
x)}
\end{equation}
```

$$\int \bar{\psi}(x)(\imath\gamma^\mu\partial_\mu - m)\psi(x) d^4x \quad (18)$$

4 Macros annexes

En plus de la large gamme de clés définies par le package, quelques macros viennent également améliorer leurs usages.

4.1 Emplacements personnalisés des différentielles

`\differentials` `\differentials`

Bien que l’option `invert-diff` existe, il est souvent souhaitable de pouvoir placer les différentielles où l’on veut. Par exemple, il est fréquent de les voir au numérateur d’une fraction lorsque celui-ci vaut 1. Pour répondre à ce besoin, le package met à disposition la macro `\differentials`. Pour les modes `default` et `nested`, la macro compose toutes les différentielles d’un coup. Avec `product`, il faudra répéter la macro entre chaque point-virgule.

```
\begin{equation}
\integral{\frac{\differentials}{x}}
\end{equation}
```

$$\int \frac{dx}{x} \quad (19)$$

4.2 Retour sur la clé *limits*

```
\NewLimitsKeyword    \NewLimitsKeyword {\mot-clé} {\bornes}
\RenewLimitsKeyword
\ProvideLimitsKeyword
\DeclareLimitsKeyword
```

Il est courant de devoir renseigner les mêmes bornes dans une intégrale. Pour faciliter leur écriture, les clés `limits(*)` acceptent, en plus des bornes explicites, un mot-clé en désignant une paire prédéfinie par l'une de ces quatre macros :

- `\NewLimitsKeyword` Crée un nouveau mot-clé et émet une erreur s'il existe déjà.
- `\RenewLimitsKeyword` Redéfinit un mot-clé et émet une erreur s'il n'existe pas déjà.
- `\ProvideLimitsKeyword` Ne crée un nouveau mot-clé que s'il n'existe pas déjà. Aucun message ne sera émis dans le cas échéant.
- `\DeclareLimitsKeyword` Crée un nouveau mot-clé quoi qu'il en soit, écrasant toute définition préalable.

Voici la liste des mots-clés déjà définis par le package et les bornes qu'ils contiennent :

<code>ab</code>	a, b
<code>unit</code>	$0, 1$
<code>real</code>	$-\infty, \infty$
<code>positive</code>	$0, \infty$
<code>negative</code>	$-\infty, 0$
<code>circle</code>	$0, 2\pi$
<code>slice</code>	$0, \pi$
<code>qcircle</code>	$0, \frac{\pi}{2}$
<code>height</code>	$0, H$
<code>radius</code>	$0, R$
<code>length</code>	$0, L$
<code>time</code>	$0, T$

remarque : Les mots-clés contiennent les *deux* bornes d'une intégrale en même temps. Ils doivent donc être précédés et/ou suivis de point-virgule.

important : Dû à l'implémentation de `\NewLimitsKeyword` et ses variantes, l'utilisateur *doit* suivre la convention d'ordre des bornes du package, même si `invert-limits` est fixé à `true`

On pourrait donc modifier l'expression d'une l'intégrale de la façon suivante :

```
\begin{equation}
  \integral[limits={radius;circle;slice}, variables={\rho, z, \phi}]{\rho z \phi}
\end{equation}
```

$$\int_0^R \int_0^{2\pi} \int_0^\pi \rho z \phi \, d\rho \, dz \, d\phi \quad (20)$$

Il est tout à fait possible de mélanger ces mots-clés avec la syntaxe plus explicite de `limits`.

```
\begin{equation}
  \int\limits[limits={time; 34, 40; height}, variables={h, \omega, t}]{h \omega t}
\end{equation}
```

$$\int_0^T \int_{34}^{40} \int_0^H h \omega t \, dh \, d\omega \, dt \quad (21)$$

4.3 Retour sur la clé *variables*

```
\NewVariableKeyword      \NewVariableKeyword {<mot-clé>} {<variables>} [<jacobien>]
\RenewVariableKeyword
\ProvideVariableKeyword
\DeclareVariableKeyword
```

De façon similaire, les mêmes groupes de variables réapparaissent souvent. On peut donc également définir des mots-clés contenant un ensemble de variables d'intégration préenregistrés. Les variantes de cette macro ont le même comportement que pour `\NewLimitsKeyword`. La seule différence est qu'il est ici possible de définir un jacobien, sous forme d'une `<csv-list>` si besoin. Son affichage est ensuite contrôlé par la clé `jacobian` expliquée précédemment. Voici la liste des mots-clés de variables déjà définis par le package, avec le jacobien pour certains :

```
cartesian    x, y, z
planar       x, y
polar        r, \theta(r)
cylindrical  r, \theta, z(r)
spherical    r, \theta, \phi(r^2, \sin \theta)
```

```
\begin{equation}
  \int\limits[triple=V, variables=spherical]{f(x, y, z)}
\end{equation}
```

$$\iiint_V f(x, y, z) \, dr \, d\theta \, d\phi \quad (22)$$

4.4 Retour sur la clé *symbol*

```

\NewSymbolKeyword    \NewSymbolKeyword {⟨clé⟩} {⟨symbole⟩}
\RenewSymbolKeyword
\ProvideSymbolKeyword
\DeclareSymbolKeyword

```

nouveau: v3.0.0

Afin de composer des intégrales indéfinies, le package offre essentiellement deux clés : `symbol` et `llimit`. Bien qu'elles soient simples d'utilisation, il demeure toujours laborieux de les écrire toutes les deux avec leurs valeurs. C'est pourquoi *intexgral* met à disposition des clés dites *raccourcies*, dont le but est de combiner l'action de sélection du symbole et des bornes. Contrairement aux deux macros précédentes, il s'agit ici de créer de toutes nouvelles clés, et non simplement des valeurs spécifiques attribuées à `symbol`. Toutes seules, ces clés modifient le symbole intégral. La valeur de ces clés correspondra elle à la borne inférieure. Voici la liste de l'ensemble des *clé-symbole* définie par le package :

<code>single</code>	symbole utilisé = <code>\int</code>
<code>double</code>	symbole utilisé = <code>\iint</code>
<code>triple</code>	symbole utilisé = <code>\iiint</code>
<code>quadruple</code>	symbole utilisé = <code>\iiiiint</code>
<code>contour</code>	symbole utilisé = <code>\oint</code>
<code>surface</code>	symbole utilisé = <code>\oiint</code>
<code>volume</code>	symbole utilisé = <code>\oiint</code>

important : Offrir la possibilité de créer soi-même une clé à la macro `\integral` représente un risque qui sera mieux expliqué dans la prochaine section. Retenez pour l'instant que toutes ces macros¹ émettront un message d'avertissement si vous tentez de créer une nouvelle *clé-symbole* portant le même nom qu'un groupe de bornes définies.

```

\begin{equation}
  \integral[contour=\mathcal{C}, diff-vec]{f(\vec r)}
\end{equation}

```

$$\oint_C f(\vec{r}) \cdot d\vec{r} \quad (23)$$

5 Syntaxe spéciale

5.1 Présentation de la syntaxe

`\integral` `\integral` [`(limits:variables(+j):mode)`] {`(intégrande)`}

nouveau: v3.0.0

En ce qui concerne les intégrales définies, l'utilisateur sera amené à employer au maximum quatre clés pour les composer : `limits`, `variables`, `mode` et `jacobian`. On peut néanmoins leur reprocher la même chose qu'avant ; écrire ces quatre clés, pouvant recevoir des arguments assez longs, va à l'encontre de l'objectif principal de ce package. Ainsi, l'utilisateur peut désigner comme argument optionnel d'`\integral`, une syntaxe dite *spéciale* qui n'est propre qu'à `intexgral`. La logique est la suivante :

- Vous spécifiez un argument *valide* de la clé `limits`
- Vous spécifiez un argument *valide* de la clé `variables`
- Vous spécifiez un argument *valide* de la clé `mode`

Et vous séparez le tout d'un deux points. Voyons un exemple pour mieux comprendre.

```
\begin{equation}
  \integral[1, 2; 4, 5:y, x:nested]{x; y}
\end{equation}
```

$$\int_1^2 x \int_4^5 y \, dy \, dx \quad (24)$$

On entend par *argument valide* le fait que toutes les formes de valeurs acceptées par les quatre clés peuvent être pareillement adoptées par la syntaxe spéciale. Cela comprend donc les mots-clés.

```
\begin{equation}
  \integral[radius;circle;scircle:spherical:product]{r;\theta;\phi}
\end{equation}
```

$$\int_0^R r \, dr \int_0^{2\pi} \theta \, d\theta \int_0^\pi \phi \, d\phi \quad (25)$$

Afin d'inclure le jacobien, il suffit d'écrire `+j` après l'argument de `variables`. Le mode peut aussi être indiqué à l'aide d'une initiale seulement.

```
\begin{equation}
  \integral[radius;circle;scircle:spherical+j:p]{;};
\end{equation}
```

$$\int_0^R r^2 \, dr \int_0^{2\pi} \sin \theta \, d\theta \int_0^\pi d\phi \quad (26)$$

Il est important de répéter que dans la configuration classique de l'argument optionnel, il n'est bien sûr pas obligatoire de renseigner les quatre clés citées. Il en va de même pour cette syntaxe. Puisque la clé `mode` n'est pas nécessaire pour `default`, cette partie peut être omise.

```
\begin{equation}
  \integral[1, 2; 3, 4:x, y]{xy}
\end{equation}
```

$$\int_1^2 \int_3^4 xy \, dx \, dy \quad (27)$$

Vous pouvez aussi toujours profiter du placement automatique du « dx » avec cette syntaxe en faisant abstraction de `variables`.

```
\begin{equation}
  \integral[1, 10]{x}
\end{equation}
```

$$\int_1^{10} x \, dx \quad (28)$$

5.2 Fonctionnement de la syntaxe

Du point de vue de l'utilisateur, il est relativement simple de choisir quelle syntaxe adopter. Toutefois, le package doit pouvoir distinguer les deux. Sans trop rentrer dans les détails¹ de l'implémentation, il est porté à votre intention que le package tente d'extraire la première clé de l'argument optionnel² et vérifie si elle existe. C'est pourquoi, si vous créez une clé avec `\NewSymbolKeyword` dont le nom pourra probablement servir de bornes d'intégrations, *intexrgal* peut faussement évaluer la nature de l'argument optionnel.

6 Paramètres annexes

`\IntegralSetup` `\IntegralSetup` {<liste de paramètres>}

nouveau: v3.0.0

Cette macro permet, sous la syntaxe <clé=valeur>, de modifier des paramètres liés à certaines clés. Toutes les assignations effectuées sont locales et la macro peut donc être placée dans un groupe si besoin. Voici un exemple de la macro avec les assignations par défaut du package :

```
\IntegralSetup
{
  defaultvar = {x},
  defaultvar* = {x},
  vectorstyle = \vec,
  domainstyle = \mathbb,
  symbolskip = {-6mu},
  hide-diff = false
}
```

1. Les plus courageux d'entre vous sont tout de même invités à lire le code source.
2. Ou du moins, le groupe de tokens qu'il pense correspondre à une clé.

defaultvar `defaultvar={⟨variables⟩}`

defaultvar* Cette clé modifie la variable d'intégration insérée automatiquement lorsque `variables` n'est pas utilisée. L'argument peut tout à fait correspondre à un mot clé défini par `\NewVariableKeyword`. La variante étoilée de la clé contrôle la variable d'intégration à composer avec `diff-vec`.

vectorstyle `vectorstyle=⟨séquence de contrôle⟩`

Cette clé modifie la macro à appliquer aux variables quand `diff-vec` est en action. Il est donc possible d'altérer le style du vecteur : gras, souligné, ou même un autre style d'un package particulier — `esvect` par exemple.

domainstyle `domainstyle=⟨séquence de contrôle⟩`

Semblablement, on peut changer la macro à appliquer pour les clés `domain(*)`.

symbolskip `symbolskip=⟨muexpr⟩`

Lorsque le mode `default` est en vigueur, le package insère un crénage entre les symboles pour légèrement les rapprocher. La dimension par défaut (-6μ) peut ainsi être ajustée selon la police mathématique en vigueur.

hide-diff `hide-diff=⟨true|false⟩`

Lorsqu'une large partie du document nécessite de ne pas inclure les différentielles, il est possible de prolonger dans la durée l'action plus ponctuelle de `variables=none`.

Historique des modifications

3.0.0 (09-12-2025)

Ajouté

- ▶ Syntaxe spéciale.
- ▶ Clés `domain*` et `mode`.
- ▶ Macros `\IntegralSetup` et `\NewSymbolKeyword`.

Supprimé

- ▶ Macros `\defaultdiff`, `\defaultdiff`, `\defaultvdiff` et `\vdiffstyle` en faveur de `\IntegralSetup`.
- ▶ Clés contrôlant symbole et borne, maintenant gérées à un plus haut niveau avec `\NewSymbolKeyword`.
- ▶ Clé `int-split`, en faveur de `mode`.
- ▶ Macro `\NewIntegralSymbol`.

Modifié

- ▶ Les noms de quelques clés (`variable` en `variables`, `lower-lim` et `upper-lim` en `llimit` et `ulimit`, `int-symb` en `symbol`, `invert-differentials` et `hide-differentials` en `invert-diff` et `hide-diff`).
- ▶ `jacobian` et `diff-star` ne nécessitent plus de valeur booléenne. Il suffit maintenant de les écrire pour activer leurs fonctionnalités.
- ▶ `hide-diff` attribué à une option locale plutôt qu'à une option de package.
- ▶ `limits-mode` attribué à une option de package plutôt qu'à une clé de macro.

v2.0.1 (13-09-2025)

Corrigé

- ▶ Problème de compatibilité entre `unicode-math` et `amssymb` selon l'ordre de chargement ([problème #2](#)).

2.0.0 (09-09-2025)

Ajouté

- ▶ Macros `\intexgralsetup`, `\defaultdiff`, `\defaultvdiff` et `\vdiffstyle`.

Modifié

- ▶ Messages d'avertissement liés aux symboles non-existants. Ils ne sont maintenant provoqués que lorsque l'intégrale est composée.

Supprimé

- ▶ Clé `diff-vec-style` en faveur de `\vdiffstyle`.
- ▶ Message d'avertissement lié à la mauvaise utilisation des points-virgules en conjonction de la clé `int-split`.

Corrigé

- ▶ Bug où l'intégrande n'était pas réinitialisée lorsque `\integral` était employée successivement dans le même groupe $\TeX$ ([détails ici](#)).

- Des restes de log *expl3* qui n'auraient pas dû être là.

1.1.0 (29-07-2025)

Ajouté

- Variantes étoilées pour les clés controlant symbole et borne (clés *single*, *contour* etc).

1.0.0 (26-07-2025) — Version initiale.

Index

Symbols

\! 751, 791, 839
 \: 752
 \{ 419, 434, 459, 461
 \} 419, 434, 459, 461
 _ . 97, 101, 105, 109, 113, 117, 122, 150, 152

B

bool commands:

\bool_if:NTF
 294, 352, 369, 409, 485,
 493, 549, 556, 572, 701, 722,
 729, 737, 752, 755, 764, 773, 778,
 783, 785, 795, 799, 802, 806,
 817, 825, 842, 1029, 1057, 1071,
 1086, 1100
 \bool_if:nTF 654, 744, 757, 830,
 844
 \bool_new:N 39, 40, 41, 201, 202,
 203, 204, 205, 206, 207, 208
 \bool_set_false:N 210, 211, 212,
 213, 214, 215, 727, 878
 \bool_set_true:N 421, 437, 442,
 726, 883, 888, 999, 1001, 1020,
 1022, 1024

C

\c 725
 \cdot 752, 764

clist commands:

\clist_item:Nn .. 338, 340, 356,
 358, 364, 366, 373, 379, 597
 \clist_map_inline:Nn 551
 \clist_new:N 217, 218
 \clist_put_right:Nn 553
 \clist_reverse:N .. 1060, 1074,
 1089, 1103
 \clist_set:Nn 335, 350, 398, 422,
 445, 471, 596, 1006, 1015, 1059,
 1073, 1088, 1102
 \l_tmpa_clist
 335, 338, 340, 350, 356,
 358, 364, 366, 373, 379, 398,
 401, 422, 426, 445, 449, 471,
 472, 596, 597, 1059, 1060, 1061,
 1073, 1074, 1075, 1088, 1089,
 1090, 1102, 1103, 1104

cs commands:

\cs_generate_variant:Nn . 172,

173, 174, 175, 176, 177, 178

\cs_if_exist:NTF 35, 913
 \cs_new:Nn 86
 \cs_new:Npn 312, 407, 590
 \cs_new_protected:Nn . 267, 331,
 669, 697, 735, 771, 815, 856
 \cs_new_protected:Npn 245, 248,
 255, 292, 307, 321, 346, 392,
 416, 431, 454, 469, 547, 594, 611
 \cs_set:Npn 1028
 \cs_set_eq:NN 788
 \cs_set_protected:Npn 739, 775,
 819

D

\d 419, 434, 459, 461
 \DeclareLimitsKeyword 1096
 \DeclareSymbolKeyword 1204
 \DeclareVariableKeyword 1138
 \differentials 150, 152, 739, 775, 788,
 819

E

exp commands:

\exp_args:Ne 959, 980
 \exp_args:NNne 621
 \exp_args:NV ... 275, 563, 579
 \exp_args:NVV 558, 574
 \exp_last_unbraced:NV ... 602
 \exp_not:N 964, 985
 \exp_not:n 538
 \ExplSyntaxOff 1281

F

\frac 1256

G

group commands:

\group_begin: 1222
 \group_end: 1230

H

hook commands:

\hook_gput_code:nnn 34

I

\iiiint 1243
 \iiint 1242
 \iint 1241
 \infty ... 375, 381, 1250, 1251, 1252
 \int 131, 199, 917, 933, 1240

int commands:

```
\int_compare:nNnTF . 495,625,
    632,671,676,925,934
\int_compare_p:nNn . . . . . 656
\int_gincr:N . . . . . 699
\int_gzero_new:N . . . . . 241
\int_new:N . . . . . 240
\int_set:Nn . . . . . 269
\int_step_inline:nn . 276,780,
    827,930
\int_use:N . . . . . 89
\integral . . . . . 1220
\IntegralSetup . . . . . 1217,1271
```

integral internal commands:

```
\__integragl_check_sequence_-
    size: . . . . . 669,
    730
\l__integragl_custom_variables_-
    bool . . . . 202,212,556,572,
    999
\l__integragl_deactivate_-
    differentials_bool 40,722,
    1001,1045
\l__integragl_default_differential_-
    tl . . . . . 194,581,622,629,
    1040
\l__integragl_default_vector_-
    differential_tl . . 195,567,
    1041
\g__integragl_differential_-
    group_keyword_prop . . 238,
    394,1004,1114,1122,1133,1140
\l__integragl_differential_-
    options_i_tl 188,559,564,
    575,580,1035
\l__integragl_differential_-
    options_ii_tl 189,483,487,
    501,515,527,538,541
\l__integragl_differential_-
    order_i_seq . . . . 229,420,
    423
\l__integragl_differential_-
    order_ii_seq . . . 230,425,
    490
\l__integragl_differential_-
    seq . . . . . 223,535,
    679,684,691,740,751,765,776,
    790,808,838,850,866
\l__integragl_differential_-
    var_i_seq 227,436,441,446,
    496,506,510
```

```
\l__integragl_differential_-
    var_ii_seq . . 228,448,520,
    524
\__integragl_differentials:nn
    . . . . . 407,537,1028
\l__integragl_display_mode_-
    str . . . . . 233,859,879,884,
    889
\l__integragl_domain_char_tl
    . . 192,958,965,979,986
\l__integragl_domain_dimension_-
    tl . . . . . 193,960,966,981,
    987
\l__integragl_domain_seq 231,
    953,954,974,975
\l__integragl_domain_style_tl
    . . . 184,964,985,1043
\__integragl_extract_differential_-
    order:n . . . . . 416,
    475
\__integragl_extract_differential_-
    var:n . . . . . 431,
    476
\__integragl_extract_first_-
    key_name:n . . . . . 594,
    1225
\__integragl_generate_integral_-
    sequence: . . . . . 267,897,
    904
\l__integragl_has_order_bool
    . . . . 207,214,421,485
\l__integragl_has_var_bool . .
    . . 208,215,437,442,493
\l__integragl_independent_-
    differential_options_tl .
    . . . . 191,456,464,484
\l__integragl_inline_symbol_-
    muskip . . . . . 235,743,
    1044
\g__integragl_integral_-
    number_int . . . . . 89,241,
    699
\__integragl_integral_preconfiguration:
    . . . . . 697,858
\l__integragl_integral_-
    sequence_int . . . 240,269,
    276
\l__integragl_integral_-
    symbol_seq 221,278,672,682,
    689,709,711,742,780,782,827,
    829
```

<code>\l__intexgral_integral_-</code> symbol_tl 187,199,257,914, 917,928,932,936	<code>__intexgral_parse_differentials:</code> 547,723
<code>\l__intexgral_integrand_seq .</code> . 220,674,677,683,690,704, 715,718,754,794,801,841	<code>__intexgral_parse_integral_-</code> limit:n 312,336, 351
<code>\l__intexgral_integrand_tl 183,</code> 706,719,725,1228	<code>__intexgral_parse_integral_-</code> symbol: 255,283, 712
<code>\l__intexgral_invert_differentials_-</code> bool 41,52,746,759,783,799, 832,846	<code>__intexgral_parse_limits:n .</code> 321,895,902
<code>\l__intexgral_invert_limits_-</code> bool .. 39,48,294,352,369, 1057,1071,1086,1100	<code>__intexgral_parse_macro_-</code> keys:n 611, 1226
<code>\l__intexgral_jacobian_bool .</code> 205,755,795,802,842,1020	<code>__intexgral_parse_variable:nn</code> .. 469,558,563,574,579
<code>\l__intexgral_jacobian_seq ..</code> 222,756,796,803,843,1010	<code>__intexgral_print_default_-</code> integral: 735,861, 865
<code>\l__intexgral_key_name_tl 182,</code> 600,606,613	<code>__intexgral_print_integral:</code> 856,1229
<code>\c__intexgral_left_bracket_tl</code> 196,377,381	<code>__intexgral_print_nested_-</code> integral: 771, 862
<code>\g__intexgral_limits_keyword_-</code> prop 237, 308,314,315,1054,1069,1084, 1098,1152,1170,1189	<code>__intexgral_print_product_-</code> integral: 815, 863
<code>\l__intexgral_limits_seq 224,</code> 326,333,348	<code>__intexgral_remove_differential_-</code> order_and_var:n 454, 477
<code>\l__intexgral_limits_style_tl</code> 42,58,60,258	<code>__intexgral_retrieve_key_-</code> name:w 590, 603
<code>\l__intexgral_lower_limit_tl</code> 185,260,296, 300,354,362,378,907,956, 957,962,977,978,983,993	<code>\c__intexgral_right_bracket_-</code> tl 197,375, 383
<code>\l__intexgral_lower_limits_-</code> seq .. 226,271,273,281,324, 337,371	<code>\l__intexgral_separate_-</code> integral_bool 203,213,701, 729,878,883,888
<code>\l__intexgral_manual_differentials_-</code> bool 201, 211,726,727,737,748,761,773, 785,806,817,834,848	<code>__intexgral_set_limits:nn ..</code> 280,292
<code>__intexgral_mathchoice:nnnn</code> 248,938	<code>__intexgral_set_limits_-</code> regular: 331, 896
<code>__intexgral_mkern:N . 245,743</code>	<code>__intexgral_set_limits_-</code> starred: 346, 903
<code>__intexgral_new_differential_-</code> group:nn .. 392,1117,1126, 1135,1144	<code>\l__intexgral_starred_differential_-</code> bool 206,409,1024, 1029
<code>__intexgral_new_limits_-</code> group:nn 173,307,1061, 1063,1075,1077,1090,1092, 1104,1106	<code>\l__intexgral_upper_limit_tl</code> .. 186,262,297,301,909

`\l_intexgral_upper_limits_-`
`seq` . 225, 272, 282, 325, 339,
370
`\l_intexgral_variable_i_-`
`clist` .. 217, 551, 576, 1006,
1015
`\l_intexgral_variable_ii_-`
`clist` 218, 553,
560
`\l_intexgral_vectorial_-`
`differential_bool` 204, 210,
549, 752, 764, 778, 825, 1022
`\l_intexgral_vectorial_-`
`differential_style_tl` 190,
554, 566, 1042
`\__intexgral_warning_msg_-`
`header`: 86, 122, 129, 136, 143,
151, 159, 165
`\intexgralsetup` 1235
iow commands:
`\iow_newline`: 22, 23, 92

K
keys commands:
`\keys_define:nn` .. 46, 871, 1038,
1157, 1174, 1193, 1206
`\keys_if_exist:nnTF` . 176, 613,
1154, 1172, 1191
`\keys_set:nn` ... 614, 637, 1218

M
`\mathbb` 32, 35, 1276
`\mathchoice` 249
msg commands:
`\msg_critical:nn` 30
`\msg_error:nnn` 1055, 1079, 1116,
1128, 1155, 1184
`\msg_line_context`: 90
`\msg_new:nnn` 19, 119, 127, 134, 141,
148, 157, 163
`\msg_new:nnnn` . 95, 99, 103, 107,
111, 115
`\msg_see_documentation_text:n`
..... 24
`\msg_warning:nn` 787, 926
`\msg_warning:nnn` 779, 826, 916,
1013, 1153, 1171, 1190
`\msg_warning:nnnn` .. 681, 688
muskip commands:
`\muskip_new:N` 235

N
`\NeedsTeXFormat` 3

`\NewDocumentCommand`
..... 1052, 1067, 1082, 1096,
1112, 1120, 1131, 1138, 1150, 1168,
1187, 1204, 1217, 1220
`\NewLimitsKeyword`
117, 122, 1052, 1249, 1250, 1251,
1252, 1253, 1254, 1255, 1256,
1257, 1258, 1259, 1260
`\NewSymbolKeyword`
.. 101, 1150, 1240, 1241, 1242,
1243, 1244, 1245, 1246
`\NewVariableKeyword` 1112, 1263, 1264,
1265, 1266, 1267
`\NewVarKeyword` 109

O
`\odif` 405, 410, 411
`\oiint` 1246
`\oint` 1245
`\oint` 1244

P
`\partial` 993
`\PassOptionsToPackage` 67, 69, 73, 75
`\phi` 1267
`\pi` 1254, 1255, 1256
`\pkginfograbProvidesExplPackage` 9
`\ProcessKeyOptions` 80
prop commands:
`\prop_get:NnNTF` ... 1003, 1054,
1084, 1114, 1133
`\prop_if_in:NnTF` 314, 1152, 1170,
1189
`\prop_item:Nn` 315
`\prop_new:N` 237, 238
`\prop_pop:NnNTF` 1069, 1098, 1122,
1140
`\prop_put:Nnn` 308, 394
`\ProvideLimitsKeyword` 1082
`\ProvideSymbolKeyword` 1187
`\ProvideVariableKeyword` 1131

Q
quark commands:
`\q_stop` 590, 603

R
regex commands:
`\regex_extract_once:nnNTF` 418,
433
`\regex_if_match:nnTF` . 172, 725
`\regex_replace_all:nnN` .. 457
`\regex_split:nnN` 616

`\RenewLimitsKeyword` 113, 1067
`\RenewSymbolKeyword` 97, 1168
`\RenewVariableKeyword` 1120
`\RenewVarKeyword` 105
`\RequirePackage` 4, 5, 35, 82

S

`\s` 419, 434, 459, 461
 scan commands:
 `\scan_stop:` . 246, 788, 939, 940,
 941, 942

seq commands:

`\seq_clear:N` 324, 325, 1125, 1143
 `\seq_count:N` . . . 272, 273, 496,
 625, 632, 656, 672, 674, 677,
 679, 682, 683, 684, 689, 690,
 691, 780, 827
 `\seq_gclear:N` 866
 `\seq_gpop_left:NN` 821
 `\seq_if_empty:NTF` 271, 709, 715
 `\seq_if_exist:NTF` 1007
 `\seq_if_in:NnTF` 519
 `\seq_item:Nn`
 281, 282, 423, 446, 490,
 506, 509, 539, 618, 636, 639,
 640, 643, 658, 782, 790, 794,
 796, 829, 837, 841, 843, 850
 `\seq_map_indexed_inline:Nn` . .
 348, 479
 `\seq_map_inline:Nn` . . 333, 954,
 975
 `\seq_new:N` 220, 221, 222, 223, 224,
 225, 226, 227, 228, 229, 230,
 231
 `\seq_pop_left:NN` 523
 `\seq_put_left:Nn` 441
 `\seq_put_right:Nn` 278, 337, 339,
 370, 371, 535, 627, 633, 711, 717
 `\seq_set_eq:NN` 1009
 `\seq_set_from_clist:NN` . . 399,
 424, 447, 472
 `\seq_set_item:Nnn` 621
 `\seq_set_split:Nnn` . 326, 634,
 703, 953, 974
 `\seq_use:Nn` 177, 740,
 742, 751, 754, 756, 765, 776, 801,
 803, 808
 `\l_tmpa_seq` 472,
 479, 539, 616, 618, 621, 625, 628,
 632, 633, 636, 639, 643
 `\l_tmpb_seq` . 634, 640, 656, 658
`\sin` 1267

str commands:

`\str_case:nnTF` 643, 859
 `\str_case_e:nnTF` 373, 379
 `\str_if_eq:nnTF` . . 174, 175, 505,
 508, 617, 1000
 `\str_if_eq_p:nn` 178, 658
 `\str_if_in:nnTF` 439
 `\str_new:N` 233
 `\str_set:Nn` 879, 884, 889
 `\str_uppercase:n` 959, 980

T

TeX and L^AT_EX 2_ε commands:

`\@ifpackagelater` 29
 `\@onlypreamble` 1235

tex commands:

`\tex_left:D` 375, 377
 `\tex_limits:D` 58
 `\tex_mathop:D` . . . 751, 791, 839
 `\tex_mathpunct:D` 357, 365
 `\tex_mkern:D` 246, 939, 940, 941,
 942
 `\tex_nolimits:D` 60
 `\tex_right:D` 381, 383

`\theta` 1265, 1266, 1267
`\times` 957, 978

tl commands:

`\c_space_tl` 89
 `\tl_clear:N` 541, 928
 `\tl_const:Nn` 196, 197
 `\tl_head:n` 959, 980
 `\tl_if_blank:nTF` 396
 `\tl_if_empty:NTF` 956, 977
 `\tl_if_empty:nTF` 473, 480, 1223
 `\tl_if_in:NnTF` 598
 `\tl_new:N` . 42, 182, 183, 184, 185,
 186, 187, 188, 189, 190, 191, 192,
 193, 194, 195
 `\tl_put_right:Nn`
 . 482, 487, 500, 514, 526, 932,
 936, 957, 962, 978, 983
 `\tl_set:Nn`
 . 296, 297, 300, 301, 323, 354,
 362, 456, 597, 600, 952, 958,
 960, 973, 979, 981, 993, 1228
 `\tl_set_eq:NN` . 58, 60, 199, 606,
 914, 917
 `\tl_tail:n` 961, 982
 `\tl_trim_spaces:n` 96, 100, 104,
 108, 112, 116, 130
 `\tl_use:N` . . . 257, 258, 378, 822

<code>\l_tmpa_tl</code>	323, 326,		987
	525, 597, 598, 603, 606, 821,	<code>\c_math_superscript_token</code>	261,
	822, 952, 953, 973, 974, 1004,		966
	1006, 1054, 1069, 1084, 1098,	<code>\token_to_str:N</code>	97, 101, 105, 109,
	1115, 1123, 1134, 1141		113, 122, 131, 150, 152
token commands:		V	
<code>\c_math_subscript_token .</code>	259,	<code>\vec</code>	1275

Implémentation

L'implémentation sera expliquée plus en détails dans une version ultérieure...

```
1 <*package>
2 <@@=intexgral>
3 \NeedsTeXFormat{LaTeX2e}
4 \RequirePackage{expl3}[2025-05-14]
5 \RequirePackage{pkginfograb}
6
7 % Package declaration
8
9 \pkginfograbProvidesExplPackage{intexgral}
10 {
11   name      = {intexgral} ,
12   author    = {Valentin Dao},
13   date      = {2025-12-24} ,
14   creation  = {2025-07-26} ,
15   version   = {3.0.0} ,
16   description = {A LaTeX package for typesetting integrals.}
17 }
18
19 \msg_new:nnn { intexgral } { expl3-too-old }
20 {
21   Your~expl3~installation~is~too~old,~
22   "intexgral"~requires~expl3~dated~2025-05-14~or~later.\iow_newline:
23   Package~loading~has~been~aborted.\iow_newline:
24   \msg_see_documentation_text:n { intexgral }
25 }
26
27 % expl3 version verification
28
29 \@ifpackagelater{expl3}{2025-05-14}{}
30 { \msg_critical:nn { intexgral } { expl3-too-old } }
31
32 % Verifies if \mathbb is defined (default domain style)
33
34 \hook_gput_code:nnn { begindocument/before } { . }
35 { \cs_if_exist:NF \mathbb { \RequirePackage{amsfonts} } }
36
37 % Definition of data types for package options
38
39 \bool_new:N \l__intexgral_invert_limits_bool
40 \bool_new:N \l__intexgral_deactivate_differentials_bool
41 \bool_new:N \l__intexgral_invert_differentials_bool
42 \tl_new:N \l__intexgral_limits_style_tl
43
44 % Package options
45
46 \keys_define:nn { intexgral }
47 {
48   invert-limits .bool_set:N = \l__intexgral_invert_limits_bool,
49   invert-limits .usage:n    = { preamble },
50   invert-limits .initial:n  = { false },
51
52   invert-diff .bool_set:N = \l__intexgral_invert_differentials_bool,
53   invert-diff .usage:n    = { preamble },
54   invert-diff .initial:n  = { false },
```

```

55
56   limits-mode .choice:,
57   limits-mode / limits .code:n =
58     { \tl_set_eq:NN \l__intexgral_limits_style_tl \tex_limits:D },
59   limits-mode / nolimits .code:n =
60     { \tl_set_eq:NN \l__intexgral_limits_style_tl \tex_nolimits:D },
61
62   limits-mode .usage:n    = { preamble },
63   limits-mode .initial:n  = { nolimits },
64
65   italic .choice:,
66     italic / true .code:n =
67       { \PassOptionsToPackage{italic=true}{derivative} },
68     italic / false .code:n =
69       { \PassOptionsToPackage{italic=false}{derivative} },
70
71   upright .choice:,
72     upright / true .code:n =
73       { \PassOptionsToPackage{upright=true}{derivative} },
74     upright / false .code:n =
75       { \PassOptionsToPackage{upright=false}{derivative} },
76 }
77
78 % Loads the package options and "derivative" package
79
80 \ProcessKeyOptions[intexgral]
81
82 \RequirePackage{derivative}
83
84 % Warning/Error/Info messages
85
86 \cs_new:Nn \__intexgral_warning_msg_header: {
87   (
88     integral~no.~
89     \int_use:N\g__intexgral_integral_number_int\c_space_tl
90     \msg_line_context:
91   )
92   \iow_newline:
93 }
94
95 \msg_new:nnnn { intexgral } { symb-key-alr-def }
96   { Symbol~key~"\tl_trim_spaces:n{#1}"~is~already~defined. }
97   { Use~\token_to_str:N \RenewSymbolKeyword\ instead. }
98
99 \msg_new:nnnn { intexgral } { symb-key-not-def }
100   { Symbol~key~"\tl_trim_spaces:n{#1}"~is~not~defined. }
101   { Use~\token_to_str:N \NewSymbolKeyword\ instead. }
102
103 \msg_new:nnnn { intexgral } { diff-group-alr-def }
104   { Differential~group~"\tl_trim_spaces:n{#1}"~is~already~defined. }
105   { Use~\token_to_str:N \RenewVarKeyword\ instead. }
106
107 \msg_new:nnnn { intexgral } { diff-group-not-def }
108   { Differential~group~"\tl_trim_spaces:n{#1}"~is~not~defined. }
109   { Use~\token_to_str:N \NewVarKeyword\ instead. }
110
111 \msg_new:nnnn { intexgral } { lim-group-alr-def }
112   { Limits~group~"\tl_trim_spaces:n{#1}"~is~already~defined. }

```

```

113 { Use~\token_to_str:N \RenewLimitsKeyword\ instead. }
114
115 \msg_new:nnnn { intexgral } { lim-group-not-def }
116 { Limits~group~"\tl_trim_spaces:n{#1}"~is~not~defined. }
117 { Use~\NewLimitsKeyword\ instead. }
118
119 \msg_new:nnn { intexgral } { key-exists-for-lim }
120 {
121   Symbol~key~already~defined~with~
122   \token_to_str:N \NewLimitsKeyword\ \__intexgral_warning_msg_header:
123   Key~"#1"~already~exists~for~predefined~limits.~
124   This~can~disrupt~the~functioning~of~the~special~syntax.
125 }
126
127 \msg_new:nnn { intexgral } { unknown-symb }
128 {
129   Unknown~integral~symbol~\__intexgral_warning_msg_header:
130   The~symbol~\tl_trim_spaces:n{#1}~has~been~replaced~with~
131   \token_to_str:N\int.
132 }
133
134 \msg_new:nnn { intexgral } { unequal-dim }
135 {
136   Inconsistent~structure~\__intexgral_warning_msg_header:
137   The~number~of~integrals~(#1),~integrands~(#2),~and~
138   differentials~(#3)~does~not~match.
139 }
140
141 \msg_new:nnn { intexgral } { no-jacobian }
142 {
143   Jacobian~unavailable~\__intexgral_warning_msg_header:
144   A~Jacobian~was~requested~for~the~"#1"~keyword,
145   ~but~none~was~declared~for~the~latter.
146 }
147
148 \msg_new:nnn { intexgral } { diff-not-sup }
149 {
150   Macro~\token_to_str:N\differentials\
151   not~supported~\__intexgral_warning_msg_header:
152   You~can't~use~\token_to_str:N\differentials\
153   with~"nested"~mode~alongside~inverted~differentials.~
154   I~will~simply~ignore~them.
155 }
156
157 \msg_new:nnn { intexgral } { diff-vec-not-supp }
158 {
159   Key~"diff-vec"~not~supported~\__intexgral_warning_msg_header:
160   You~can't~use~"diff-vec"~with~"#1"~mode.~I~will~simply~ignore~this~key.
161 }
162
163 \msg_new:nnn { intexgral } { use-glyph-instead }
164 {
165   Consider~using~the~dedicated~glyph~\__intexgral_warning_msg_header:
166   The~key~"nint"~was~used~with~fewer~than~
167   5~regular~integral~signs.
168 }
169
170 % Argument specifiers variants

```

```

171
172 \cs_generate_variant:Nn \regex_if_match:nnTF { nVTF }
173 \cs_generate_variant:Nn \__intexgral_new_limits_group:nn { nV }
174 \cs_generate_variant:Nn \str_if_eq:nnTF { neTF }
175 \cs_generate_variant:Nn \str_if_eq:nnF { neF }
176 \cs_generate_variant:Nn \keys_if_exist:nnTF { nVTF }
177 \cs_generate_variant:Nn \seq_use:Nn { Ne }
178 \cs_generate_variant:Nn \str_if_eq_p:nn { en }
179
180 % DATA TYPES
181
182 \tl_new:N \l__intexgral_key_name_tl
183 \tl_new:N \l__intexgral_integrand_tl
184 \tl_new:N \l__intexgral_domain_style_tl
185 \tl_new:N \l__intexgral_lower_limit_tl
186 \tl_new:N \l__intexgral_upper_limit_tl
187 \tl_new:N \l__intexgral_integral_symbol_tl
188 \tl_new:N \l__intexgral_differential_options_i_tl
189 \tl_new:N \l__intexgral_differential_options_ii_tl
190 \tl_new:N \l__intexgral_vectorial_differential_style_tl
191 \tl_new:N \l__intexgral_independent_differential_options_tl
192 \tl_new:N \l__intexgral_domain_char_tl
193 \tl_new:N \l__intexgral_domain_dimension_tl
194 \tl_new:N \l__intexgral_default_differential_tl
195 \tl_new:N \l__intexgral_default_vector_differential_tl
196 \tl_const:Nn \c__intexgral_left_bracket_tl { [ }
197 \tl_const:Nn \c__intexgral_right_bracket_tl { ] }
198
199 \tl_set_eq:NN \l__intexgral_integral_symbol_tl \int
200
201 \bool_new:N \l__intexgral_manual_differentials_bool
202 \bool_new:N \l__intexgral_custom_variables_bool
203 \bool_new:N \l__intexgral_separate_integral_bool
204 \bool_new:N \l__intexgral_vectorial_differential_bool
205 \bool_new:N \l__intexgral_jacobian_bool
206 \bool_new:N \l__intexgral_starred_differential_bool
207 \bool_new:N \l__intexgral_has_order_bool
208 \bool_new:N \l__intexgral_has_var_bool
209
210 \bool_set_false:N \l__intexgral_vectorial_differential_bool
211 \bool_set_false:N \l__intexgral_manual_differentials_bool
212 \bool_set_false:N \l__intexgral_custom_variables_bool
213 \bool_set_false:N \l__intexgral_separate_integral_bool
214 \bool_set_false:N \l__intexgral_has_order_bool
215 \bool_set_false:N \l__intexgral_has_var_bool
216
217 \clist_new:N \l__intexgral_variable_i_clist
218 \clist_new:N \l__intexgral_variable_ii_clist
219
220 \seq_new:N \l__intexgral_integrand_seq
221 \seq_new:N \l__intexgral_integral_symbol_seq
222 \seq_new:N \l__intexgral_jacobian_seq
223 \seq_new:N \l__intexgral_differential_seq
224 \seq_new:N \l__intexgral_limits_seq
225 \seq_new:N \l__intexgral_upper_limits_seq
226 \seq_new:N \l__intexgral_lower_limits_seq
227 \seq_new:N \l__intexgral_differential_var_i_seq
228 \seq_new:N \l__intexgral_differential_var_ii_seq

```

```

229 \seq_new:N \l__intexgral_differential_order_i_seq
230 \seq_new:N \l__intexgral_differential_order_ii_seq
231 \seq_new:N \l__intexgral_domain_seq
232
233 \str_new:N \l__intexgral_display_mode_str
234
235 \muskip_new:N \l__intexgral_inline_symbol_muskip
236
237 \prop_new:N \g__intexgral_limits_keyword_prop
238 \prop_new:N \g__intexgral_differential_group_keyword_prop
239
240 \int_new:N \l__intexgral_integral_sequence_int
241 \int_gzero_new:N \g__intexgral_integral_number_int
242
243 % TeX macros adapted to expl3 syntax
244
245 \cs_new_protected:Npn \__intexgral_mkern:N #1
246 { \tex_mkern:D #1 \scan_stop: }
247
248 \cs_new_protected:Npn \__intexgral_mathchoice:nnnn #1#2#3#4
249 { \mathchoice{#1}{#2}{#3}{#4} }
250
251 % SYMBOL
252
253 % General form of the symbol (glyph, limits mode, lower limit, upper limit)
254
255 \cs_new_protected:Npn \__intexgral_parse_integral_symbol:
256 {
257   \tl_use:N \l__intexgral_integral_symbol_tl
258   \tl_use:N \l__intexgral_limits_style_tl
259   \c_math_subscript_token
260   { \l__intexgral_lower_limit_tl }
261   \c_math_superscript_token
262   { \l__intexgral_upper_limit_tl }
263 }
264
265 % Generates the symbols after <limits> is executed
266
267 \cs_new_protected:Nn \__intexgral_generate_integral_sequence:
268 {
269   \int_set:Nn \l__intexgral_integral_sequence_int
270   {
271     \seq_if_empty:NTF \l__intexgral_lower_limits_seq
272     { \seq_count:N \l__intexgral_upper_limits_seq }
273     { \seq_count:N \l__intexgral_lower_limits_seq }
274   }
275   \exp_args:NV
276   \int_step_inline:nn \l__intexgral_integral_sequence_int
277   {
278     \seq_put_right:Nn \l__intexgral_integral_symbol_seq
279     {
280       \__intexgral_set_limits:nn
281       { \seq_item:Nn \l__intexgral_lower_limits_seq { ##1 } }
282       { \seq_item:Nn \l__intexgral_upper_limits_seq { ##1 } }
283       \__intexgral_parse_integral_symbol:
284     }
285   }
286 }

```

```

287
288 % LIMITS
289
290 % Inverts the limit order convention
291
292 \cs_new_protected:Npn \__intexgral_set_limits:nn #1#2
293 {
294   \bool_if:NTF \l__intexgral_invert_limits_bool
295   {
296     \tl_set:Nn \l__intexgral_lower_limit_tl { #2 }
297     \tl_set:Nn \l__intexgral_upper_limit_tl { #1 }
298   }
299   {
300     \tl_set:Nn \l__intexgral_lower_limit_tl { #1 }
301     \tl_set:Nn \l__intexgral_upper_limit_tl { #2 }
302   }
303 }
304
305 % Creates a keyword containing predefined limits
306
307 \cs_new_protected:Npn \__intexgral_new_limits_group:nn #1#2
308 { \prop_put:Nnn \g__intexgral_limits_keyword_prop { #1 } { #2 } }
309
310 % Verifies if the argument corresponds to a valid keyword
311
312 \cs_new:Npn \__intexgral_parse_integral_limit:n #1
313 {
314   \prop_if_in:NnTF \g__intexgral_limits_keyword_prop { #1 }
315   { \prop_item:Nn \g__intexgral_limits_keyword_prop { #1 } }
316   { #1 }
317 }
318
319 % Splits the argument of the key to prepare for parsing
320
321 \cs_new_protected:Npn \__intexgral_parse_limits:n #1
322 {
323   \tl_set:Nn \l_tmpa_tl { #1 }
324   \seq_clear:N \l__intexgral_lower_limits_seq
325   \seq_clear:N \l__intexgral_upper_limits_seq
326   \seq_set_split:NnV \l__intexgral_limits_seq { ; } \l_tmpa_tl
327 }
328
329 % Limits under regular form
330
331 \cs_new_protected:Nn \__intexgral_set_limits_regular:
332 {
333   \seq_map_inline:Nn \l__intexgral_limits_seq
334   {
335     \clist_set:Ne \l_tmpa_clist
336     { \__intexgral_parse_integral_limit:n { ##1 } }
337     \seq_put_right:Ne \l__intexgral_lower_limits_seq
338     { \clist_item:Nn \l_tmpa_clist { 1 } }
339     \seq_put_right:Ne \l__intexgral_upper_limits_seq
340     { \clist_item:Nn \l_tmpa_clist { 2 } }
341   }
342 }
343
344 % Limits under interval form

```

```

345
346 \cs_new_protected:Npn \__intexgral_set_limits_starred:
347 {
348   \seq_map_indexed_inline:Nn \l__intexgral_limits_seq
349   {
350     \clist_set:Nc \l_tmpa_clist
351       { \__intexgral_parse_integral_limit:n { ##2 } }
352     \bool_if:NTF \l__intexgral_invert_limits_bool
353     {
354       \tl_set:Nc \l__intexgral_lower_limit_tl
355       {
356         \clist_item:Nn \l_tmpa_clist { 2 }
357         \tex_mathpunct:D,
358         \clist_item:Nn \l_tmpa_clist { 1 }
359       }
360     }
361     {
362       \tl_set:Nc \l__intexgral_lower_limit_tl
363       {
364         \clist_item:Nn \l_tmpa_clist { 1 }
365         \tex_mathpunct:D,
366         \clist_item:Nn \l_tmpa_clist { 2 }
367       }
368     }
369     \bool_if:NTF \l__intexgral_invert_limits_bool
370     { \seq_put_right:Nc \l__intexgral_upper_limits_seq { }
371       { \seq_put_right:Nc \l__intexgral_lower_limits_seq { }
372         {
373           \str_case_e:nnF { \clist_item:Nn \l_tmpa_clist { 1 } } {
374             {
375               { -\infty } { \tex_left:D \c__intexgral_right_bracket_tl }
376             }
377             { \tex_left:D \c__intexgral_left_bracket_tl }
378           \tl_use:N \l__intexgral_lower_limit_tl
379           \str_case_e:nnF { \clist_item:Nn \l_tmpa_clist { 2 } } {
380             {
381               { +\infty } { \tex_right:D \c__intexgral_left_bracket_tl }
382             }
383             { \tex_right:D \c__intexgral_right_bracket_tl }
384           }
385         }
386       }
387     }
388 % VARIABLES
389
390 % Creates a keyword containing predefined variables
391
392 \cs_new_protected:Npn \__intexgral_new_differential_group:nnn #1#2#3
393 {
394   \prop_put:Nnn \g__intexgral_differential_group_keyword_prop
395     { #1 } { #2 }
396   \tl_if_blank:nF { #3 }
397   {
398     \clist_set:Nn \l_tmpa_clist { #3 }
399     \seq_set_from_clist:cN
400       { g__intexgral_#1_jacobian_seq }
401     \l_tmpa_clist
402   }

```



```

403 }
404
405 % expl3 equivalent of \odif(*)
406
407 \cs_new:Npn \__intexgral_differentials:nn #1#2
408 {
409     \bool_if:NTF \l__intexgral_starred_differential_bool
410     { \odif*[#1]{#2} }
411     { \odif[#1]{#2} }
412 }
413
414 % Extracts the whole "order" <keyval>
415
416 \cs_new_protected:Npn \__intexgral_extract_differential_order:n #1
417 {
418     \regex_extract_once:nnNT
419     { order\s*=\s*\{?\s*(\d+(?:\s*,\s*\d+)*\s*)\}? }
420     { #1 } \l__intexgral_differential_order_i_seq
421     { \bool_set_true:N \l__intexgral_has_order_bool }
422     \clist_set:Ne \l_tmpa_clist
423     { \seq_item:Nn \l__intexgral_differential_order_i_seq { 2 } }
424     \seq_set_from_clist:NN
425     \l__intexgral_differential_order_ii_seq
426     \l_tmpa_clist
427 }
428
429 % Extracts the whole "var" <keyval>
430
431 \cs_new_protected:Npn \__intexgral_extract_differential_var:n #1
432 {
433     \regex_extract_once:nnNTF
434     { var\s*=\s*(none|all|\{\s*(\d+(?:\s*,\s*\d+)*\s*)\}) }
435     { #1 }
436     \l__intexgral_differential_var_i_seq
437     { \bool_set_true:N \l__intexgral_has_var_bool }
438     {
439         \str_if_in:nnT { #1 } { var }
440         {
441             \seq_put_left:Nn \l__intexgral_differential_var_i_seq { var }
442             \bool_set_true:N \l__intexgral_has_var_bool
443         }
444     }
445     \clist_set:Ne \l_tmpa_clist
446     { \seq_item:Nn \l__intexgral_differential_var_i_seq { 3 } }
447     \seq_set_from_clist:NN
448     \l__intexgral_differential_var_ii_seq
449     \l_tmpa_clist
450 }
451
452 % Extracts all keys other than "order" and "var"
453
454 \cs_new_protected:Npn \__intexgral_remove_differential_order_and_var:n #1
455 {
456     \tl_set:Nn \l__intexgral_independent_differential_options_tl { #1 }
457     \regex_replace_all:nnN
458     {
459         order\s*=\s*\{?\s*\d+(\s*,\s*\d+)*\s*\}?\s*,?
460         |

```

```

461     [^~]var(?:\s*=\s*(?:none|all|\{\s*\d+(?:\s*,\s*\d+)*\s*\}))?,?
462 }
463 { }
464 \l__intexgral_independent_differential_options_tl
465 }
466
467 % Reassembles the variables
468
469 \cs_new_protected:Npn \__intexgral_parse_variable:nn #1#2
470 {
471   \clist_set:Nn \l_tmpa_clist { #2 }
472   \seq_set_from_clist:NN \l_tmpa_seq \l_tmpa_clist
473   \tl_if_empty:nF { #1 }
474   {
475     \__intexgral_extract_differential_order:n { #1 }
476     \__intexgral_extract_differential_var:n { #1 }
477     \__intexgral_remove_differential_order_and_var:n { #1 }
478   }
479   \seq_map_indexed_inline:Nn \l_tmpa_seq {
480     \tl_if_empty:nF { #1 }
481     {
482       \tl_put_right:NV
483         \l__intexgral_differential_options_ii_tl
484         \l__intexgral_independent_differential_options_tl
485       \bool_if:NT \l__intexgral_has_order_bool
486       {
487         \tl_put_right:Ne \l__intexgral_differential_options_ii_tl
488           {
489             ,order=
490             \seq_item:Nn \l__intexgral_differential_order_ii_seq { ##1 }
491           }
492       }
493       \bool_if:NT \l__intexgral_has_var_bool
494       {
495         \int_compare:nNnTF
496           { \seq_count:N \l__intexgral_differential_var_i_seq }
497           =
498           { 1 }
499           {
500             \tl_put_right:Nn
501               \l__intexgral_differential_options_ii_tl
502               { var }
503           }
504           {
505             \str_if_eq:neF { none }
506             { \seq_item:Nn \l__intexgral_differential_var_i_seq { 2 } }
507             {
508               \str_if_eq:neTF { all }
509               { \seq_item:Nn
510                 \l__intexgral_differential_var_i_seq
511                 { 2 }
512               }
513               {
514                 \tl_put_right:Nn
515                   \l__intexgral_differential_options_ii_tl
516                   { var }
517               }
518             }

```

```

519         \seq_if_in:NnT
520         \l__intexgral_differential_var_ii_seq
521         { ##1 }
522         {
523             \seq_pop_left:NN
524             \l__intexgral_differential_var_ii_seq
525             \l_tmpa_tl
526             \tl_put_right:Nn
527             \l__intexgral_differential_options_ii_tl
528             { ,var }
529         }
530     }
531 }
532 }
533 }
534 }
535 \seq_put_right:Ne \l__intexgral_differential_seq
536 {
537     \__intexgral_differentials:nn
538     { \exp_not:V \l__intexgral_differential_options_ii_tl }
539     { \seq_item:Nn \l_tmpa_seq { ##1 } }
540 }
541 \tl_clear:N \l__intexgral_differential_options_ii_tl
542 }
543 }
544
545 % Executes all previous macros to push tokens into the variable sequence
546
547 \cs_new_protected:Npn \__intexgral_parse_differentials:
548 {
549     \bool_if:NTF \l__intexgral_vectorial_differential_bool
550     {
551         \clist_map_inline:Nn \l__intexgral_variable_i_clist
552         {
553             \clist_put_right:Nn \l__intexgral_variable_ii_clist
554             { \l__intexgral_vectorial_differential_style_tl{##1} }
555         }
556         \bool_if:NTF \l__intexgral_custom_variables_bool
557         {
558             \exp_args:NVV \__intexgral_parse_variable:nn
559             \l__intexgral_differential_options_i_tl
560             \l__intexgral_variable_ii_clist
561         }
562         {
563             \exp_args:NV \__intexgral_parse_variable:nn
564             \l__intexgral_differential_options_i_tl
565             {
566                 \l__intexgral_vectorial_differential_style_tl
567                 { \l__intexgral_default_vector_differential_tl }
568             }
569         }
570     }
571     {
572         \bool_if:NTF \l__intexgral_custom_variables_bool
573         {
574             \exp_args:NVV \__intexgral_parse_variable:nn
575             \l__intexgral_differential_options_i_tl
576             \l__intexgral_variable_i_clist

```

```

577     }
578     {
579         \exp_args:NV \__intexgral_parse_variable:nn
580         \l__intexgral_differential_options_i_tl
581         { \l__intexgral_default_differential_tl }
582     }
583 }
584 }
585
586 % SPECIAL SYNTAX
587
588 % Cuts off the value of a key
589
590 \cs_new:Npn \__intexgral_retrieve_key_name:w #1=#2\q_stop { #1 }
591
592 % Extracts first key name of optional argument
593
594 \cs_new_protected:Npn \__intexgral_extract_first_key_name:n #1
595 {
596     \clist_set:Nn \l_tmpa_clist { #1 }
597     \tl_set:Ne \l_tmpa_tl { \clist_item:Nn \l_tmpa_clist { 1 } }
598     \tl_if_in:NnTF \l_tmpa_tl { = }
599     {
600         \tl_set:Ne \l__intexgral_key_name_tl
601         {
602             \exp_last_unbraced:NV
603             \__intexgral_retrieve_key_name:w \l_tmpa_tl \q_stop
604         }
605     }
606     { \tl_set_eq:NN \l__intexgral_key_name_tl \l_tmpa_tl }
607 }
608
609 % Sets the keys as usual, or sets them up manually if special syntax is in use
610
611 \cs_new_protected:Npn \__intexgral_parse_macro_keys:n #1
612 {
613     \keys_if_exist:nVTF { integral } \l__intexgral_key_name_tl
614     { \keys_set:nn { integral } { #1 } }
615     {
616         \regex_split:nnN { : } { #1 } \l_tmpa_seq
617         \str_if_eq:eeTF
618         { \seq_item:Nn \l_tmpa_seq { 2 } }
619         { +j }
620         {
621             \exp_args:NNne \seq_set_item:Nnn \l_tmpa_seq { 2 }
622             { \l__intexgral_default_differential_tl+j }
623         }
624         {
625             \int_compare:nNnT { \seq_count:N \l_tmpa_seq } < { 2 }
626             {
627                 \seq_put_right:NV
628                 \l_tmpa_seq
629                 \l__intexgral_default_differential_tl
630             }
631         }
632         \int_compare:nNnT { \seq_count:N \l_tmpa_seq } < { 3 }
633         { \seq_put_right:Nn \l_tmpa_seq { d } }
634         \seq_set_split:Nne \l_tmpb_seq

```

```

635     { + }
636     { \seq_item:Nn \l_tmpa_seq { 2 } }
637 \keys_set:ne { integral }
638 {
639     limits      = { \seq_item:Nn \l_tmpa_seq { 1 } },
640     variables    = { \seq_item:Nn \l_tmpb_seq { 1 } },
641     mode         =
642     {
643         \str_case:enF { \seq_item:Nn \l_tmpa_seq { 3 } }
644         {
645             { d      } { default }
646             { n      } { nested  }
647             { p      } { product  }
648             { default } { default }
649             { nested } { nested  }
650             { product } { product }
651         }
652         { default }
653     },
654     \bool_if:nT
655     {
656         \int_compare_p:nNn { \seq_count:N \l_tmpb_seq } > { 1 }
657         &&
658         \str_if_eq_p:en { \seq_item:Nn \l_tmpb_seq { 2 } } { j }
659     }
660     { jacobian }
661 }
662 }
663 }
664
665 % PRELIMINARY CONFIGURATIONS
666
667 % Verifies that <limits> and <variables> (and <integrand>) have the same size
668
669 \cs_new_protected:Nn \__intexgral_check_sequence_size:
670 {
671     \int_compare:nNnTF
672     { \seq_count:N \l__intexgral_integral_symbol_seq }
673     =
674     { \seq_count:N \l__intexgral_integrand_seq }
675     {
676         \int_compare:nNnF
677         { \seq_count:N \l__intexgral_integrand_seq }
678         =
679         { \seq_count:N \l__intexgral_differential_seq }
680         {
681             \msg_warning:nneee { intexgral } { unequal-dim }
682             { \seq_count:N \l__intexgral_integral_symbol_seq }
683             { \seq_count:N \l__intexgral_integrand_seq }
684             { \seq_count:N \l__intexgral_differential_seq }
685         }
686     }
687 }
688 \msg_warning:nneee { intexgral } { unequal-dim }
689 { \seq_count:N \l__intexgral_integral_symbol_seq }
690 { \seq_count:N \l__intexgral_integrand_seq }
691 { \seq_count:N \l__intexgral_differential_seq }
692 }

```

```

693 }
694
695 % Performs preparatory actions before executing the main macro
696
697 \cs_new_protected:Nn \__intexgral_integral_preconfiguration:
698 {
699   \int_gincr:N \g__intexgral_integral_number_int
700
701   \bool_if:NT \l__intexgral_separate_integral_bool
702   {
703     \seq_set_split:NnV
704       \l__intexgral_integrand_seq
705       { ; }
706       \l__intexgral_integrand_tl
707   }
708
709   \seq_if_empty:NT \l__intexgral_integral_symbol_seq
710   {
711     \seq_put_right:Nn \l__intexgral_integral_symbol_seq
712       { \__intexgral_parse_integral_symbol: }
713   }
714
715   \seq_if_empty:NT \l__intexgral_integrand_seq
716   {
717     \seq_put_right:NV
718       \l__intexgral_integrand_seq
719       \l__intexgral_integrand_tl
720   }
721
722   \bool_if:NF \l__intexgral_deactivate_differentials_bool
723   { \__intexgral_parse_differentials: }
724
725   \regex_if_match:nVTF { \c{differentials} } \l__intexgral_integrand_tl
726   { \bool_set_true:N \l__intexgral_manual_differentials_bool }
727   { \bool_set_false:N \l__intexgral_manual_differentials_bool }
728
729   \bool_if:NT \l__intexgral_separate_integral_bool
730   { \__intexgral_check_sequence_size: }
731 }
732
733 % Default mode
734
735 \cs_new_protected:Nn \__intexgral_print_default_integral:
736 {
737   \bool_if:NT \l__intexgral_manual_differentials_bool
738   {
739     \cs_set_protected:Npn \differentials
740       { \seq_use:Nn \l__intexgral_differential_seq { } }
741   }
742   \seq_use:Nn \l__intexgral_integral_symbol_seq
743     { \__intexgral_mkern:N \l__intexgral_inline_symbol_muskip }
744   \bool_if:nT
745   {
746     \l__intexgral_invert_differentials_bool
747     &&
748     !\l__intexgral_manual_differentials_bool
749   }
750   {

```

```

751     \seq_use:Nn \l__intexgral_differential_seq { } \tex_mathop:D{} \!
752     \bool_if:NT \l__intexgral_vectorial_differential_bool { \cdot \: }
753   }
754   \seq_use:Nn \l__intexgral_integrand_seq { }
755   \bool_if:NT \l__intexgral_jacobian_bool
756     { \seq_use:Nn \l__intexgral_jacobian_seq { } }
757   \bool_if:nT
758     {
759       !\l__intexgral_invert_differentials_bool
760       &&
761       !\l__intexgral_manual_differentials_bool
762     }
763     {
764       \bool_if:NT \l__intexgral_vectorial_differential_bool { \cdot }
765       \seq_use:Nn \l__intexgral_differential_seq { }
766     }
767   }
768
769 % Nested mode
770
771 \cs_new_protected:Nn \__intexgral_print_nested_integral:
772 {
773   \bool_if:NT \l__intexgral_manual_differentials_bool
774     {
775       \cs_set_protected:Npn \differentials
776         { \seq_use:Nn \l__intexgral_differential_seq { } }
777     }
778   \bool_if:NT \l__intexgral_vectorial_differential_bool
779     { \msg_warning:nnn { intexgral } { diff-vec-not-supp } { nested } }
780   \int_step_inline:nn { \seq_count:N \l__intexgral_integral_symbol_seq }
781     {
782       \seq_item:Nn \l__intexgral_integral_symbol_seq { ##1 }
783       \bool_if:NTF \l__intexgral_invert_differentials_bool
784         {
785           \bool_if:NT \l__intexgral_manual_differentials_bool
786             {
787               \msg_warning:nn { intexgral } { diff-not-sup }
788               \cs_set_eq:NN \differentials \scan_stop:
789             }
790           \seq_item:Nn \l__intexgral_differential_seq { ##1 }
791           \tex_mathop:D{} \!
792         }
793         {
794           \seq_item:Nn \l__intexgral_integrand_seq { ##1 }
795           \bool_if:NT \l__intexgral_jacobian_bool
796             { \seq_item:Nn \l__intexgral_jacobian_seq { ##1 } }
797         }
798     }
799   \bool_if:NTF \l__intexgral_invert_differentials_bool
800     {
801       \seq_use:Nn \l__intexgral_integrand_seq {}
802       \bool_if:NT \l__intexgral_jacobian_bool
803         { \seq_use:Nn \l__intexgral_jacobian_seq { } }
804     }
805     {
806       \bool_if:NF \l__intexgral_manual_differentials_bool
807         {
808           \seq_use:Nn \l__intexgral_differential_seq {}

```

```

809     }
810   }
811 }
812
813 % Product mode
814
815 \cs_new_protected:Nn \__intexgral_print_product_integral:
816 {
817   \bool_if:NT \l__intexgral_manual_differentials_bool
818   {
819     \cs_set_protected:Npn \differentials
820     {
821       \seq_gpop_left:NN \l_tmpa_tl
822       \tl_use:N \l_tmpa_tl
823     }
824   }
825   \bool_if:NT \l__intexgral_vectorial_differential_bool
826   { \msg_warning:nnn { intexgral } { diff-vec-not-supp } { product } }
827   \int_step_inline:nn { \seq_count:N \l__intexgral_integral_symbol_seq }
828   {
829     \seq_item:Nn \l__intexgral_integral_symbol_seq { ##1 }
830     \bool_if:nT
831     {
832       \l__intexgral_invert_differentials_bool
833       &&
834       !\l__intexgral_manual_differentials_bool
835     }
836     {
837       \seq_item:Nn
838       \l__intexgral_differential_seq { ##1 }
839       \tex_mathop:D{} \!
840     }
841     \seq_item:Nn \l__intexgral_integrand_seq { ##1 }
842     \bool_if:NT \l__intexgral_jacobian_bool
843     { \seq_item:Nn \l__intexgral_jacobian_seq { ##1 } }
844     \bool_if:nT
845     {
846       !\l__intexgral_invert_differentials_bool
847       &&
848       !\l__intexgral_manual_differentials_bool
849     }
850     { \seq_item:Nn \l__intexgral_differential_seq { ##1 } }
851   }
852 }
853
854 % General macro for typesetting
855
856 \cs_new_protected:Nn \__intexgral_print_integral:
857 {
858   \__intexgral_integral_preconfiguration:
859   \str_case:VnF \l__intexgral_display_mode_str
860   {
861     { default } { \__intexgral_print_default_integral: }
862     { nested } { \__intexgral_print_nested_integral: }
863     { product } { \__intexgral_print_product_integral: }
864   }
865   { \__intexgral_print_default_integral: }
866   \seq_gclear:N \l__intexgral_differential_seq

```



```

867 }
868
869 % KEYS
870
871 \keys_define:nn { integral } {
872
873   % LIMITS
874
875   mode .choice:,
876     mode / default .code:n =
877     {
878       \bool_set_false:N \l__intexgral_separate_integral_bool
879       \str_set:Nn \l__intexgral_display_mode_str { default }
880     },
881     mode / nested .code:n =
882     {
883       \bool_set_true:N \l__intexgral_separate_integral_bool
884       \str_set:Nn \l__intexgral_display_mode_str { nested }
885     },
886     mode / product .code:n =
887     {
888       \bool_set_true:N \l__intexgral_separate_integral_bool
889       \str_set:Nn \l__intexgral_display_mode_str { product }
890     },
891     mode .default:n = { default },
892
893   limits .code:n =
894   {
895     \__intexgral_parse_limits:n { #1 }
896     \__intexgral_set_limits_regular:
897     \__intexgral_generate_integral_sequence:
898   },
899
900   limits* .code:n =
901   {
902     \__intexgral_parse_limits:n { #1 }
903     \__intexgral_set_limits_starred:
904     \__intexgral_generate_integral_sequence:
905   },
906
907   llimit .tl_set:N = \l__intexgral_lower_limit_tl,
908
909   ulimit .tl_set:N = \l__intexgral_upper_limit_tl,
910
911   symbol .code:n =
912   {
913     \cs_if_exist:NTF #1
914     { \tl_set_eq:NN \l__intexgral_integral_symbol_tl #1 }
915     {
916       \msg_warning:nnn { intexgral } { unknown-symb } { #1 }
917       \tl_set_eq:NN \l__intexgral_integral_symbol_tl \int
918     }
919   },
920
921   % SYMBOL SHORTCUT
922
923   nint .code:n =
924   {

```

```

925 \int_compare:nNt { #1 } < { 5 }
926 { \msg_warning:nn { intexgral } { use-glyph-instead } }
927
928 \tl_clear:N \l__intexgral_integral_symbol_tl
929
930 \int_step_inline:nn { #1 }
931 {
932   \tl_put_right:Nn \l__intexgral_integral_symbol_tl
933     { \int }
934   \int_compare:nNt { ##1 } < { #1 }
935   {
936     \tl_put_right:Nn \l__intexgral_integral_symbol_tl
937       {
938         \__intexgral_mathchoice:nnnn
939         { \tex_mkern:D -12mu \scan_stop: }
940         { \tex_mkern:D -8mu \scan_stop: }
941         { \tex_mkern:D -4mu \scan_stop: }
942         { \tex_mkern:D -2mu \scan_stop: }
943       }
944     }
945   }
946 },
947
948 % LIMITS SHORTCUTS
949
950 domain .code:n =
951 {
952   \tl_set:Nn \l_tmpa_tl { #1 }
953   \seq_set_split:NnV \l__intexgral_domain_seq { * } \l_tmpa_tl
954   \seq_map_inline:Nn \l__intexgral_domain_seq
955   {
956     \tl_if_empty:NF \l__intexgral_lower_limit_tl
957     { \tl_put_right:Nn \l__intexgral_lower_limit_tl { \times } }
958     \tl_set:Ne \l__intexgral_domain_char_tl
959     { \exp_args:Ne \str_uppercase:n { \tl_head:n { ##1 } } }
960     \tl_set:Ne \l__intexgral_domain_dimension_tl
961     { \tl_tail:n { ##1 } }
962     \tl_put_right:Ne \l__intexgral_lower_limit_tl
963     {
964       \exp_not:N \l__intexgral_domain_style_tl
965       { \l__intexgral_domain_char_tl }
966       \c_math_superscript_token { \l__intexgral_domain_dimension_tl }
967     }
968   }
969 },
970
971 domain* .code:n =
972 {
973   \tl_set:Nn \l_tmpa_tl { #1 }
974   \seq_set_split:NnV \l__intexgral_domain_seq { * } \l_tmpa_tl
975   \seq_map_inline:Nn \l__intexgral_domain_seq
976   {
977     \tl_if_empty:NF \l__intexgral_lower_limit_tl
978     { \tl_put_right:Nn \l__intexgral_lower_limit_tl { \times } }
979     \tl_set:Ne \l__intexgral_domain_char_tl
980     { \exp_args:Ne \str_uppercase:n { \tl_head:n { ##1 } } }
981     \tl_set:Ne \l__intexgral_domain_dimension_tl
982     { \tl_tail:n { ##1 } }

```

```

983         \tl_put_right:N\l__intexgral_lower_limit_tl
984         {
985             \exp_not:N \l__intexgral_domain_style_tl
986             { \l__intexgral_domain_char_tl }
987             \c_math_subscript_token { \l__intexgral_domain_dimension_tl }
988         }
989     }
990 },
991
992 boundary .code:n =
993     { \tl_set:Nn \l__intexgral_lower_limit_tl { \partial #1 } },
994
995 % VARIABLES
996
997 variables .code:n =
998     {
999         \bool_set_true:N \l__intexgral_custom_variables_bool
1000         \str_if_eq:nnTF { #1 } { none }
1001         { \bool_set_true:N \l__intexgral_deactivate_differentials_bool }
1002         {
1003             \prop_get:NnNTF
1004             \g__intexgral_differential_group_keyword_prop { #1 } \l_tmpa_tl
1005             {
1006                 \clist_set:NV \l__intexgral_variable_i_clist \l_tmpa_tl
1007                 \seq_if_exist:cTF { g__intexgral_#1_jacobian_seq }
1008                 {
1009                     \seq_set_eq:Nc
1010                     \l__intexgral_jacobian_seq
1011                     { g__intexgral_#1_jacobian_seq }
1012                 }
1013                 { \msg_warning:nnn { intexgral } { no-jacobian } { #1 } }
1014             }
1015             { \clist_set:Nn \l__intexgral_variable_i_clist { #1 } }
1016         }
1017     },
1018
1019 jacobian .code:n =
1020     { \bool_set_true:N \l__intexgral_jacobian_bool },
1021 diff-vec .code:n =
1022     { \bool_set_true:N \l__intexgral_vectorial_differential_bool },
1023 diff-star .code:n =
1024     { \bool_set_true:N \l__intexgral_starred_differential_bool },
1025
1026 diff-symb .code:n =
1027     {
1028         \cs_set:Npn \__intexgral_differentials:nn ##1##2 {
1029             \bool_if:NTF \l__intexgral_starred_differential_bool
1030             { #1*{##1}{##2} }
1031             { #1[##1]{##2} }
1032         }
1033     },
1034
1035 diff-options .tl_set:N = \l__intexgral_differential_options_i_tl,
1036 }
1037
1038 \keys_define:nn { IntegralSetup }
1039 {
1040     defaultvar .tl_set:N = \l__intexgral_default_differential_tl,

```

```

1041     defaultvar* .tl_set:N = \l__intexgral_default_vector_differential_tl,
1042     vectorstyle .tl_set:N = \l__intexgral_vectorial_differential_style_tl,
1043     domainstyle .tl_set:N = \l__intexgral_domain_style_tl,
1044     symbolskip .tl_set:N = \l__intexgral_inline_symbol_muskip,
1045     hide-diff .bool_set:N = \l__intexgral_deactivate_differentials_bool,
1046   }
1047
1048 % END USER MACROS
1049
1050 % Limits keywords
1051
1052 \NewDocumentCommand\NewLimitsKeyword{ m m }
1053 {
1054   \prop_get:NnNTF \g__intexgral_limits_keyword_prop { #1 } \l_tmpa_tl
1055   { \msg_error:nnn { intexgral } { lim-group-alr-def } { #1 } }
1056   {
1057     \bool_if:NTF \l__intexgral_invert_limits_bool
1058     {
1059       \clist_set:Nn \l_tmpa_clist { #2 }
1060       \clist_reverse:N \l_tmpa_clist
1061       \__intexgral_new_limits_group:nV { #1 } \l_tmpa_clist
1062     }
1063     { \__intexgral_new_limits_group:nn { #1 } { #2 } }
1064   }
1065 }
1066
1067 \NewDocumentCommand\RenewLimitsKeyword{ m m }
1068 {
1069   \prop_pop:NnNTF \g__intexgral_limits_keyword_prop { #1 } \l_tmpa_tl
1070   {
1071     \bool_if:NTF \l__intexgral_invert_limits_bool
1072     {
1073       \clist_set:Nn \l_tmpa_clist { #2 }
1074       \clist_reverse:N \l_tmpa_clist
1075       \__intexgral_new_limits_group:nV { #1 } \l_tmpa_clist
1076     }
1077     { \__intexgral_new_limits_group:nn { #1 } { #2 } }
1078   }
1079   { \msg_error:nnn { intexgral } { lim-group-not-def } }
1080 }
1081
1082 \NewDocumentCommand\ProvideLimitsKeyword{ m m }
1083 {
1084   \prop_get:NnNF \g__intexgral_limits_keyword_prop { #1 } \l_tmpa_tl
1085   {
1086     \bool_if:NTF \l__intexgral_invert_limits_bool
1087     {
1088       \clist_set:Nn \l_tmpa_clist { #2 }
1089       \clist_reverse:N \l_tmpa_clist
1090       \__intexgral_new_limits_group:nV { #1 } \l_tmpa_clist
1091     }
1092     { \__intexgral_new_limits_group:nn { #1 } { #2 } }
1093   }
1094 }
1095
1096 \NewDocumentCommand\DeclareLimitsKeyword{ m m }
1097 {
1098   \prop_pop:NnNT \g__intexgral_limits_keyword_prop { #1 } \l_tmpa_tl

```

```

1099     {
1100         \bool_if:NTF \l__intexgral_invert_limits_bool
1101         {
1102             \clist_set:Nn \l_tmpa_clist { #2 }
1103             \clist_reverse:N \l_tmpa_clist
1104             \__intexgral_new_limits_group:nV { #1 } \l_tmpa_clist
1105         }
1106         { \__intexgral_new_limits_group:nn { #1 } { #2 } }
1107     }
1108 }
1109
1110 % Variable keywords
1111
1112 \NewDocumentCommand\NewVariableKeyword{ m m o }
1113 {
1114     \prop_get:NnNTF \g__intexgral_differential_group_keyword_prop { #1 }
1115     \l_tmpa_tl
1116     { \msg_error:nnn { intexgral } { diff-group-alr-def } { #1 } }
1117     { \__intexgral_new_differential_group:nnn { #1 } { #2 } { #3 } }
1118 }
1119
1120 \NewDocumentCommand\RenewVariableKeyword{ m m o }
1121 {
1122     \prop_pop:NnNTF \g__intexgral_differential_group_keyword_prop { #1 }
1123     \l_tmpa_tl
1124     {
1125         \seq_clear:c { g__intexgral_#1_jacobian_seq }
1126         \__intexgral_new_differential_group:nnn { #1 } { #2 } { #3 }
1127     }
1128     { \msg_error:nnn { intexgral } { diff-group-not-def } { #1 } }
1129 }
1130
1131 \NewDocumentCommand\ProvideVariableKeyword{ m m o }
1132 {
1133     \prop_get:NnNF \g__intexgral_differential_group_keyword_prop { #1 }
1134     \l_tmpa_tl
1135     { \__intexgral_new_differential_group:nnn { #1 } { #2 } { #3 } }
1136 }
1137
1138 \NewDocumentCommand\DeclareVariableKeyword{ m m o }
1139 {
1140     \prop_pop:NnNT \g__intexgral_differential_group_keyword_prop { #1 }
1141     \l_tmpa_tl
1142     {
1143         \seq_clear:c { g__intexgral_#1_jacobian_seq }
1144         \__intexgral_new_differential_group:nnn { #1 } { #2 } { #3 }
1145     }
1146 }
1147
1148 % Symbol keys
1149
1150 \NewDocumentCommand\NewSymbolKeyword{ m m }
1151 {
1152     \prop_if_in:NnT \g__intexgral_limits_keyword_prop { #1 }
1153     { \msg_warning:nnn { intexgral } { key-exists-for-lim } { #1 } }
1154     \keys_if_exist:nnTF { integral } { #1 }
1155     { \msg_error:nnn { intexgral } { symb-key-alr-def } { #1 } }
1156     {

```

```

1157     \keys_define:nn { integral }
1158     {
1159         #1 .meta:n =
1160         {
1161             symbol=#2,
1162             llimit=##1
1163         },
1164     }
1165 }
1166 }
1167
1168 \NewDocumentCommand\RenewSymbolKeyword{ m m }
1169 {
1170     \prop_if_in:NnT \g__intexgral_limits_keyword_prop { #1 }
1171     { \msg_warning:nnn { intexgral } { key-exists-for-lim } { #1 } }
1172     \keys_if_exist:nnTF { integral } { #1 }
1173     {
1174         \keys_define:nn { integral }
1175         {
1176             #1 .undefine:
1177             #1 .meta:n =
1178             {
1179                 symbol=#2,
1180                 llimit=##1
1181             }
1182         }
1183     }
1184     { \msg_error:nnn { intexgral } { symb-key-not-def } }
1185 }
1186
1187 \NewDocumentCommand\ProvideSymbolKeyword{ m m }
1188 {
1189     \prop_if_in:NnT \g__intexgral_limits_keyword_prop { #1 }
1190     { \msg_warning:nnn { intexgral } { key-exists-for-lim } { #1 } }
1191     \keys_if_exist:nnF { integral } { #1 }
1192     {
1193         \keys_define:nn { integral }
1194         {
1195             #1 .meta:n =
1196             {
1197                 symbol=#2,
1198                 llimit=##1
1199             },
1200         }
1201     }
1202 }
1203
1204 \NewDocumentCommand\DeclareSymbolKeyword{ m m }
1205 {
1206     \keys_define:nn { integral }
1207     {
1208         #1 .undefine:
1209         #1 .meta:n =
1210         {
1211             symbol=#2,
1212             llimit=##1
1213         },
1214     }

```

```

1215 }
1216
1217 \NewDocumentCommand\IntegralSetup{ m }
1218 { \keys_set:nn { IntegralSetup } { #1 } }
1219
1220 \NewDocumentCommand\integral{ 0{} m }
1221 {
1222   \group_begin:
1223   \tl_if_empty:nF { #1 }
1224   {
1225     \__intexgral_extract_first_key_name:n { #1 }
1226     \__intexgral_parse_macro_keys:n { #1 }
1227   }
1228   \tl_set:Nn \l__intexgral_integrand_tl { #2 }
1229   \__intexgral_print_integral:
1230   \group_end:
1231 }
1232
1233 % Preamble only
1234
1235 \@onlypreamble\intexgralsetup
1236
1237 % Included keywords
1238
1239 % Symbol keywords
1240 \NewSymbolKeyword{single}      {\int}
1241 \NewSymbolKeyword{double}     {\iint}
1242 \NewSymbolKeyword{triple}     {\iiint}
1243 \NewSymbolKeyword{quadruple}  {\iiiiint}
1244 \NewSymbolKeyword{contour}    {\oint}
1245 \NewSymbolKeyword{surface}    {\oiint}
1246 \NewSymbolKeyword{volume}     {\oiint}
1247
1248 % Limits keywords
1249 \NewLimitsKeyword{ab}         {a, b}
1250 \NewLimitsKeyword{real}       {-\infty, \infty}
1251 \NewLimitsKeyword{positive}   {0, \infty}
1252 \NewLimitsKeyword{negative}   {-\infty, 0}
1253 \NewLimitsKeyword{unit}       {0, 1}
1254 \NewLimitsKeyword{circle}     {0, 2\pi}
1255 \NewLimitsKeyword{scircle}    {0, \pi}
1256 \NewLimitsKeyword{qcircle}    {0, \frac{\pi}{2}}
1257 \NewLimitsKeyword{height}     {0, H}
1258 \NewLimitsKeyword{radius}     {0, R}
1259 \NewLimitsKeyword{length}     {0, L}
1260 \NewLimitsKeyword{time}       {0, T}
1261
1262 % Variable keywords
1263 \NewVariableKeyword{cartesian} {x, y, z}
1264 \NewVariableKeyword{planar}    {x, y}
1265 \NewVariableKeyword{polar}     {r, \theta} [r]
1266 \NewVariableKeyword{cylindrical} {r, \theta, z} [r]
1267 \NewVariableKeyword{spherical} {r, \theta, \phi} [r^2, \sin\theta]
1268
1269 % Default parameters
1270
1271 \IntegralSetup
1272 {

```

```

1273     defaultvar = {x},
1274     defaultvar* = {r},
1275     vectorstyle = \vec,
1276     domainstyle = \mathbb,
1277     symbolskip = {-6mu},
1278     hide-diff = false
1279 }
1280
1281 \ExplSyntaxOff

```